

Designmodell

Modul: T4INF2003 – Software Engineering I

Kurs: TINF2024

Dozentin: Charlotte Chichlow

Semester: Frühjahr 2026

Projekt: Entwicklung eines Online Aufgaben-Planungssystems

Designmodell.....	1
1. Überblick.....	2
2. Systemübersicht und Architektur.....	2
2.1 Produktbeschreibung.....	2
2.2 Architekturentscheidung: Client-Server.....	2
2.3 Schichten und Technologieentscheidungen.....	3
3. Statisches Modell.....	3
3.1 Paketstruktur.....	4
3.2 Klassendiagramm.....	6
3.2.1 ER-Schema der Datenbank.....	10
3.3 Architektur / Services.....	11
4. Dynamisches Modell.....	12
4.1 Use-Case-Diagramme des dynamischen Modells.....	12
4.1.1 Authentication Use Cases.....	12
4.1.2 Organization Use Cases.....	13
4.1.3 Planning Context Use Cases.....	14
4.1.4 Profile Use Cases.....	15
4.2 Sequenzdiagramm: Benutzer registrieren und anmelden.....	15
4.3 Sequenzdiagramm: Aufgabe erstellen.....	16
4.4 Sequenzdiagramm: Arbeitsplan generieren.....	17
4.4.1 Beschreibung des Planungsalgorithmus.....	18
4.5 Sequenzdiagramm: Aufgabe bearbeiten.....	20
4.6 Identifizierte Methoden je Klasse.....	21
5. API-Spezifikation.....	22
6. Design Patterns.....	23
7. Frontend-Struktur.....	24
8. Deployment und Entwicklungsinfrastruktur.....	25

1. Überblick

Das System „TeaPot“ ist eine browserbasierte Webanwendung, die Einzelpersonen und Teams dabei unterstützt, Aufgaben nicht nur zu erfassen, sondern auch automatisiert und ressourcenbasiert zu planen. Im Gegensatz zu klassischen To-Do-Listen oder Kalender Applikationen überbrückt TeaPot die Lücke zwischen statischem Aufgabenmanagement und aktivem Kalendermanagement: Ein Planungsalgorithmus legt Aufgaben anhand von Priorität, Deadline, Abhängigkeiten und individuellem Arbeitsprofil selbstständig in freie Zeitfenster ein.

2. Systemübersicht und Architektur

2.1 Produktbeschreibung

Das System richtet sich an Einzelpersonen und Teams, die Aufgaben strukturiert planen und koordinieren wollen. Benutzer können Aufgaben mit Prioritäten (Low, Medium, High), Deadlines, Zeitschätzungen und Abhängigkeiten konfigurieren sowie über ein persönliches Arbeitsprofil (tägliche Arbeitszeit, maximale Belastung) die Rahmenbedingungen für einen automatisch generierten Arbeitsplan definieren.

2.2 Architekturentscheidung: Client-Server

Das System setzt auf eine **drei-schichtige Client-Server-Architektur** mit klarer Trennung von Frontend (React), Backend (ASP.NET Core / C#) und Datenpersistenz (PostgreSQL). Die Kommunikation zwischen Frontend und Backend erfolgt ausschließlich über eine REST-API. Diese Architektur wurde gewählt, weil:

- **Separation of Concerns:** Frontend und Backend können unabhängig voneinander entwickelt, getestet und gewartet werden.
- **Skalierbarkeit:** Mehrere Clients können denselben Backend-Server nutzen, ohne dass Änderungen am Server notwendig werden.
- **Sicherheit:** Authentifizierung wird zentral über Auth0 verwaltet.
- **Testbarkeit:** Backend-Logik und Frontend-Komponenten lassen sich isoliert testen.

2.3 Schichten und Technologieentscheidungen

Schicht	Technologie	Begründung
Frontend	React 19 + Vite + TypeScript; FullCalendar; Tailwind CSS	Moderne React-Version mit schnellem Build-Tool (Vite), Typsicherheit durch TypeScript, flexible UI-Gestaltung mit Tailwind und FullCalendar für Kalenderfunktionen
Backend	.NET 10 / ASP.NET Core 10	Aktuelle .NET/ASP.NET Core-Version für Performance und Langzeit-Support;
Datenbank	PostgreSQL (Produktiv) + EF Core 10 + Npgsql	PostgreSQL als robuste, skalierbare Produktiv-Datenbank; EF Core 10 mit Npgsql-Provider für komfortablen Datenzugriff
API-Kommunikation	REST (JSON) mit OpenAPI/Swagger	Standardisierte, weit verbreitete Schnittstelle; JSON als gängiges Austauschformat; OpenAPI/Swagger für dokumentierte, leicht testbare APIs.
ORM	Entity Framework Core 10 (Npgsql-Provider)	Moderne ORM-Version mit guter PostgreSQL-Unterstützung; abstrahiert SQL und ermöglicht eine saubere Repository- oder Service-Schicht.
Authentifizierung	Auth0 (JWT-Validierung im Backend, Auth0 Management-Client bei Bedarf)	Zentrale, skalierbare Identity-Plattform; JWTs werden serverseitig geprüft, wodurch die API zustandslos bleibt; Management-Client erlaubt Benutzer- und Rollenverwaltung über Auth0-APIs.

3. Statisches Modell

Hinweis zum Scope: In der ersten Iteration des Designmodells werden nur die für den MVP relevanten Kernobjekte (User, UserTask, WorkProfile, TimeInterval) detailliert modelliert und in den Sequenzdiagrammen verwendet. Die im Pflichtenheft definierten Objekte **Organisation**, **Arbeitsplan**, **Aufgabenblock** und deren Assoziationen sind fachlich zentral, werden aber für Iteration 1 nur im statischen Modell ergänzt, um die spätere Erweiterung zu ermöglichen. Im Fokus stehen zunächst die reinen Nutzer- und Planungsfunktionen.

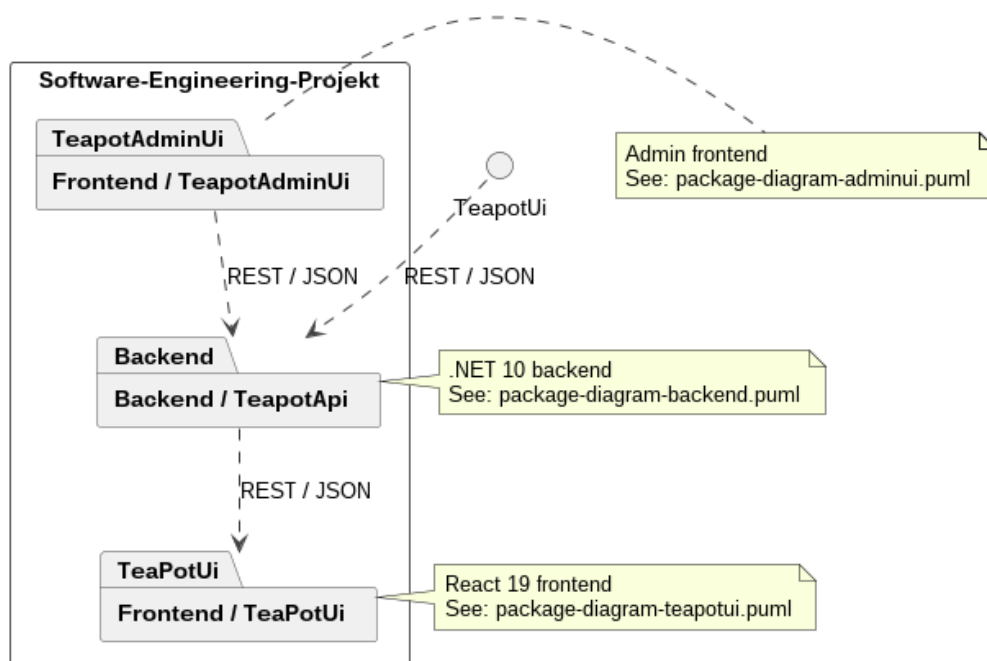
3.1 Paketstruktur

Die Darstellung erfolgt aus Gründen der Übersichtlichkeit in mehreren Teilabbildungen.

Gesamtübersicht

Das Paketdiagramm auf oberster Ebene zeigt die Makroarchitektur des Systems *TeaPot*. Das System besteht aus einem Backend (*TeapotApi*) sowie zwei Frontends: der eigentlichen Nutzeroberfläche (*TeaPotUi*) und einer Administrationsoberfläche (*TeapotAdminUi*). Die Kommunikation zwischen Frontend und Backend erfolgt über REST/JSON-Schnittstellen. Dadurch wird eine klare Trennung zwischen Präsentationsschicht und Geschäftslogik erreicht.

TeaPot - Package Diagram Overview



Backend

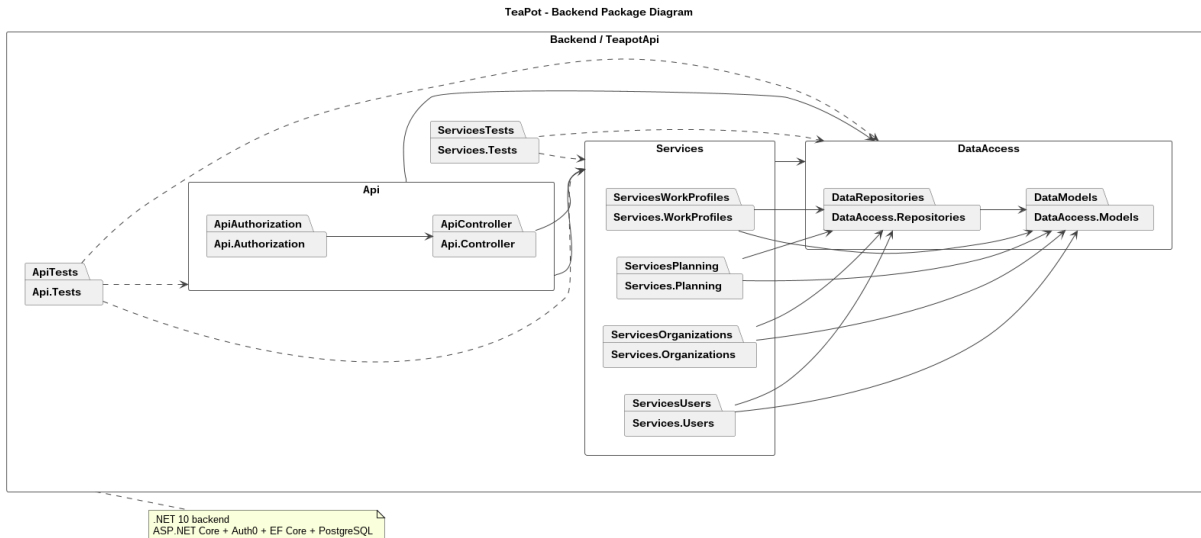
Das Backend ist in die Bereiche API, Services und DataAccess unterteilt.

Die API-Schicht stellt die Endpunkte bereit und verarbeitet Anfragen.

Die Services-Schicht enthält die zentrale Geschäftslogik, etwa für Benutzer, Organisationen, Planung und Arbeitsprofile.

Die DataAccess-Schicht kapselt den Zugriff auf Persistenzobjekte und Repositories.

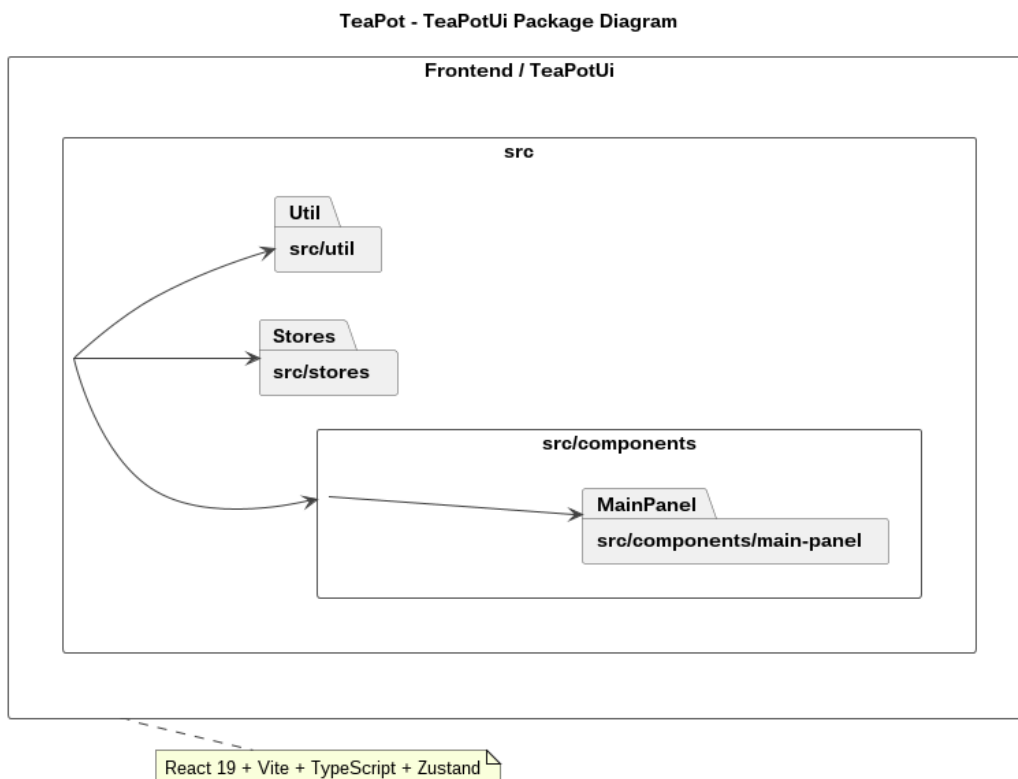
Zusätzlich zeigen die Testpakete, dass sowohl API als auch Services getrennt getestet werden.



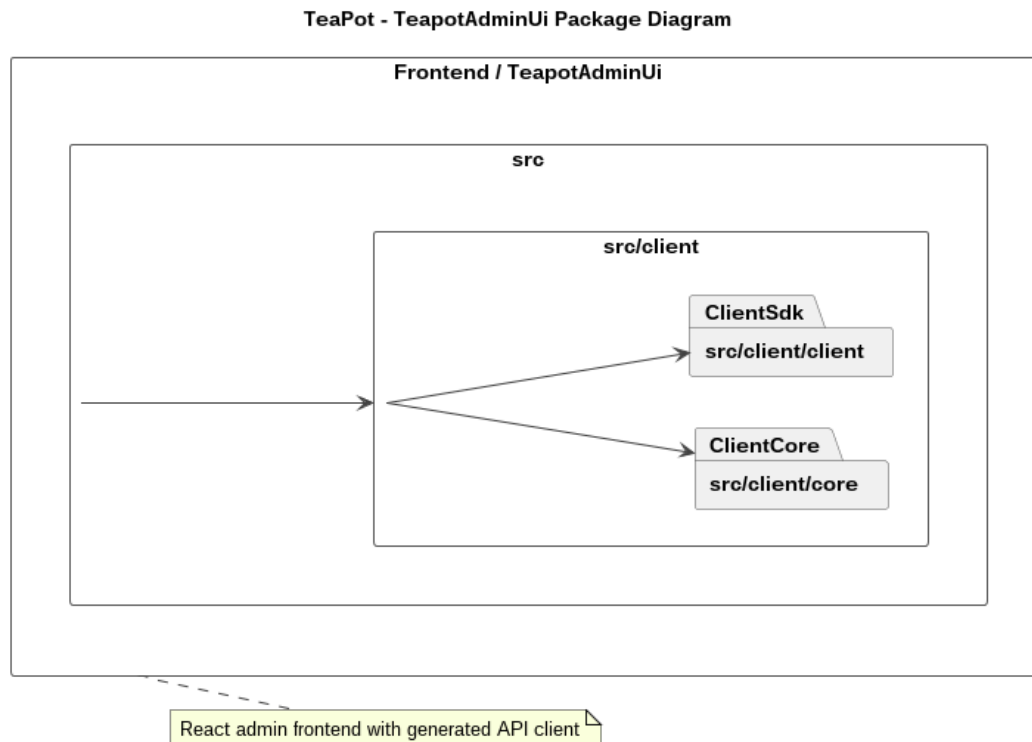
Frontend (User GUI)

Das Paketdiagramm der Benutzeroberfläche zeigt die Struktur des regulären Frontends. Im Paket `src` befinden sich die Bereiche Komponenten, Stores und Hilfsfunktionen/Utilities. Die Komponenten bilden die sichtbare Oberfläche, die Stores verwalten den Anwendungszustand, und die Utility-Schicht kapselt technische Zugriffe wie API-Kommunikation.

Damit folgt das Frontend einer klaren Trennung zwischen UI, State-Management und Servicezugriff.



Frontend (System Admin GUI)



3.2 Klassendiagramm

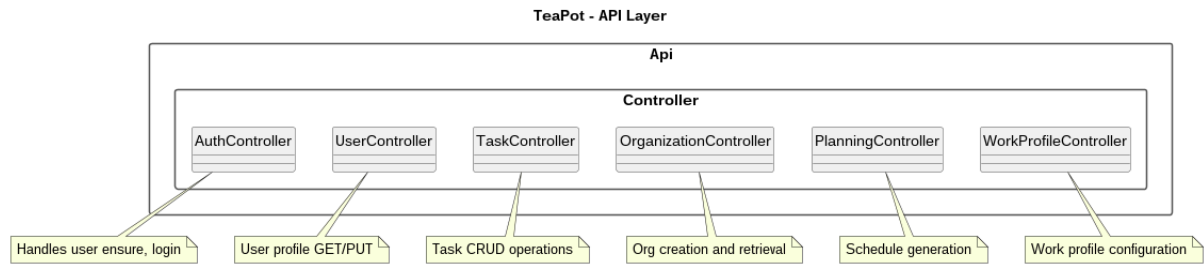
Das dargestellte Klassendiagramm bildet die Brücke zwischen der statischen Datenhaltung im Paket *DataAccess* und der dynamischen Planungslogik in *Logic / Planning*. Im Zentrum steht die Erweiterung der *UserTask*-Entität um spezifische Parameter der *Critical Path Analysis* (CPA) sowie Persistenz-Attribute für fixierte Aufgabenblöcke .

Über das *IUserTaskPlanner*-Interface wird eine entkoppelte Architektur realisiert, die es dem *SchedulingAlgorithm* erlaubt, komplexe Abhängigkeiten unter Berücksichtigung von Intensitätsprofilen und zeitlichen Puffern in einen konkreten *WorkPlan* zu überführen .

API-Layer

Das Klassendiagramm des API-Layers zeigt die zentralen Controller-Klassen des Backends. Jeder Controller übernimmt einen klar abgegrenzten Verantwortungsbereich, z. B. Authentifizierung, Benutzerverwaltung, Aufgabenverwaltung, Organisationsverwaltung, Planung und Arbeitsprofile.

Die Controller bilden damit die Einstiegspunkte in das System und leiten eingehende HTTP-Anfragen an die darunterliegenden Services weiter.

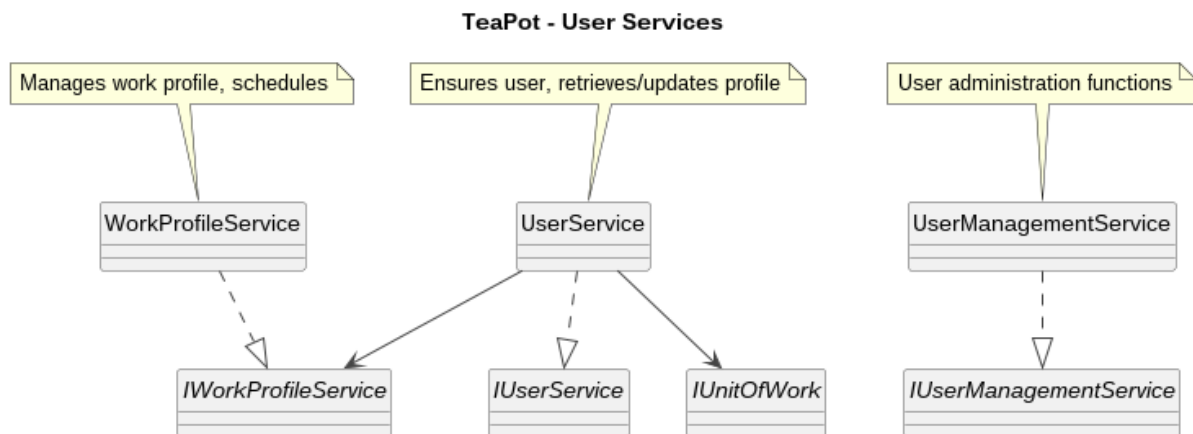


User Services

Dieses Diagramm beschreibt die Services rund um Benutzer und Arbeitsprofile. Die Interfaces definieren jeweils die Verträge, während die konkreten Serviceklassen deren Implementierung enthalten.

Der UserService übernimmt die Verwaltung und Aktualisierung von Benutzerinformationen und nutzt dafür unter anderem ein IUnitOfWork sowie den IWorkProfileService.

Das Diagramm macht deutlich, dass die Geschäftslogik über lose gekoppelte Service-Schnittstellen organisiert ist.



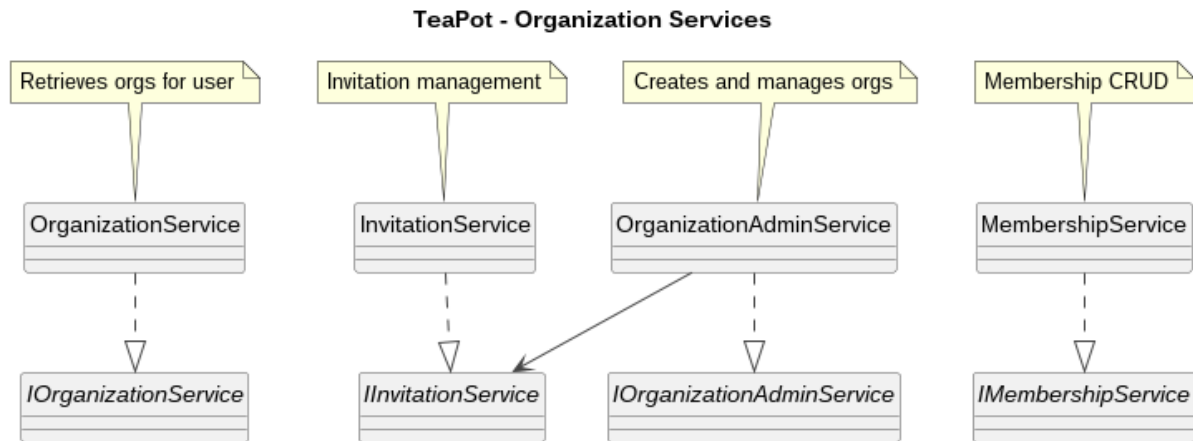
Organization Services

Das Diagramm zeigt die Service-Struktur für Organisationen, Mitgliedschaften und Einladungen.

Die Funktionen sind auf mehrere spezialisierte Services verteilt: allgemeine Organisationsfunktionen, administrative Verwaltung, Mitgliedschaftsverwaltung und Einladungshandling.

Besonders sichtbar ist, dass administrative Funktionen auf weitere Services wie den Einladungsservice zurückgreifen.

So wird die Organisationslogik in fachlich getrennte Verantwortungsbereiche aufgeteilt.

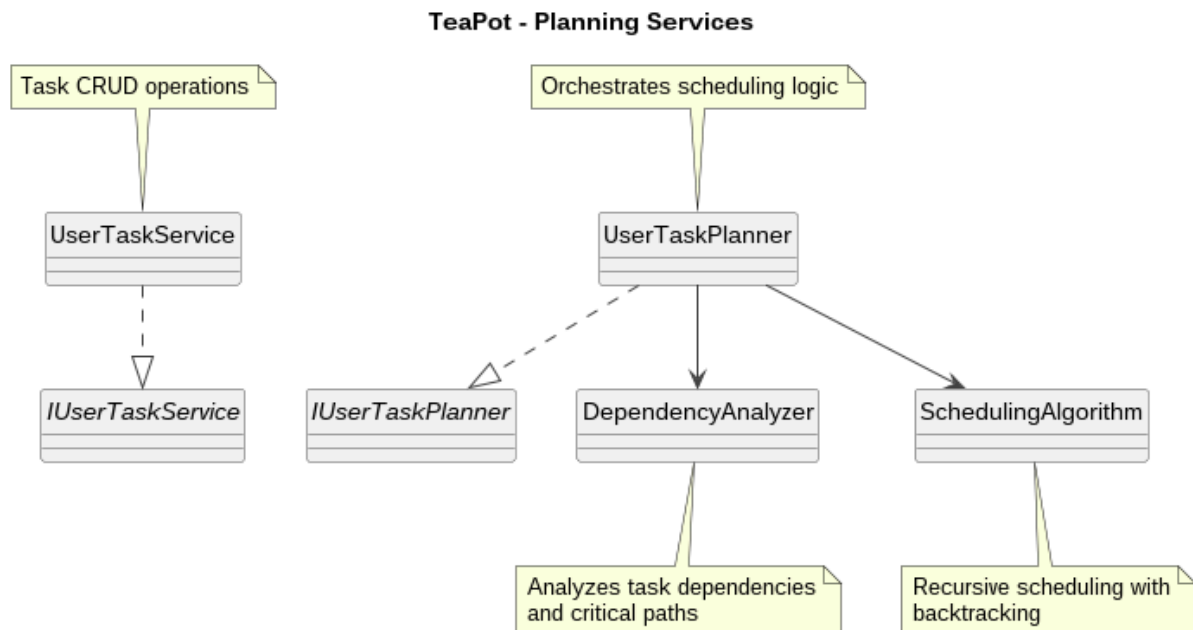


Planning Services

Dieses Diagramm beschreibt die Planungslogik des Systems.

Der UserTaskService verwaltet Aufgaben, während der UserTaskPlanner die eigentliche Planerstellung orchestriert.

Dabei greift der Planner auf Hilfskomponenten wie DependencyAnalyzer und SchedulingAlgorithm zurück.



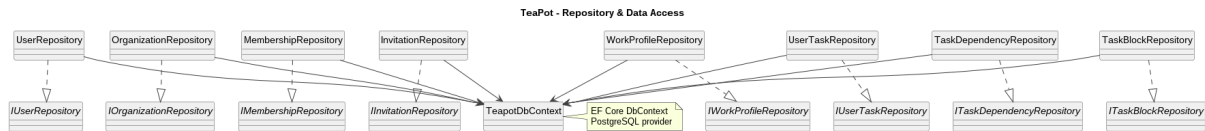
Repository & Data Access

Das Data-Access-Diagramm zeigt die technische Persistenzschicht.

Zentral ist der TeapotDbContext, über den die Repositories auf die Datenbank zugreifen.

Für jede wesentliche Domäne existiert ein eigenes Repository, z. B. für Benutzer, Organisationen, Mitgliedschaften, Einladungen, Aufgaben und Abhängigkeiten.

Dadurch wird der Datenzugriff strukturiert gekapselt und von der Geschäftslogik getrennt.

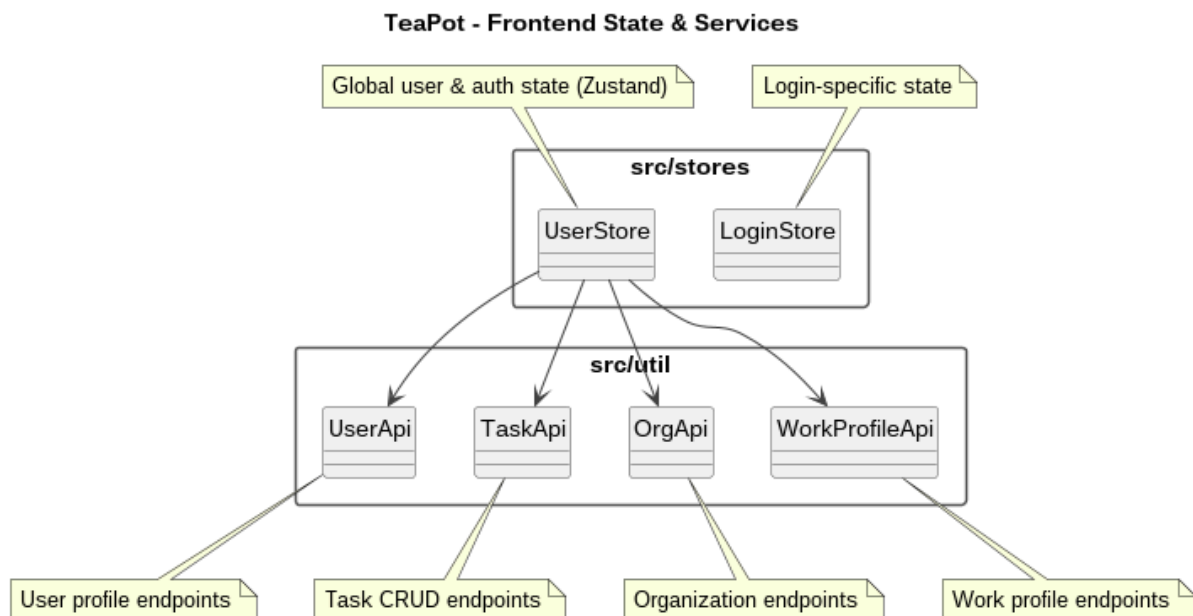


Frontend State & Services

Dieses Diagramm beschreibt die Zustandsverwaltung und die API-Anbindung im Frontend. Die Stores, insbesondere UserStore und LoginStore, verwalten den globalen Zustand der Anwendung.

Für die Kommunikation mit dem Backend existieren spezialisierte API-Klassen wie UserApi, TaskApi, OrgApi und WorkProfileApi.

Damit wird deutlich, dass die Frontend-Logik sauber zwischen Zustand und Backend-Kommunikation getrennt ist.



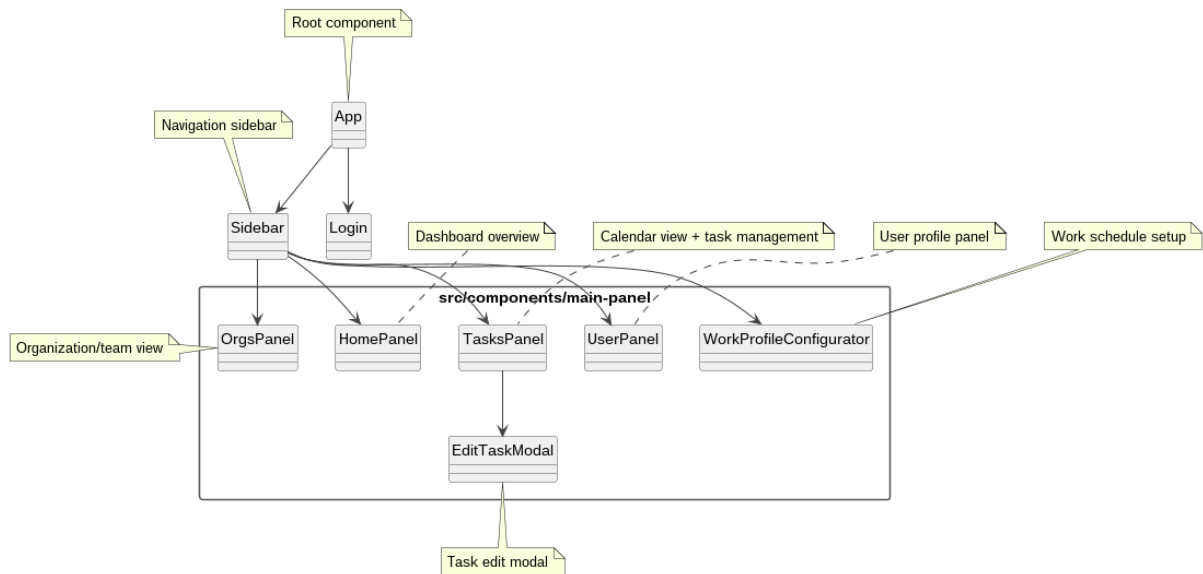
Frontend UI Components

Das Klassendiagramm der UI-Komponenten zeigt die wesentlichen Bausteine der Benutzeroberfläche.

App ist die Wurzel der Anwendung, Sidebar dient der Navigation, und einzelne Panels kapseln die jeweiligen Funktionsbereiche wie Dashboard, Aufgaben, Organisationen, Benutzerprofil und Arbeitsprofil-Konfiguration.

Damit wird die Oberfläche als modular aufgebaute Komponentenstruktur dargestellt.

TeaPot - Frontend UI Components



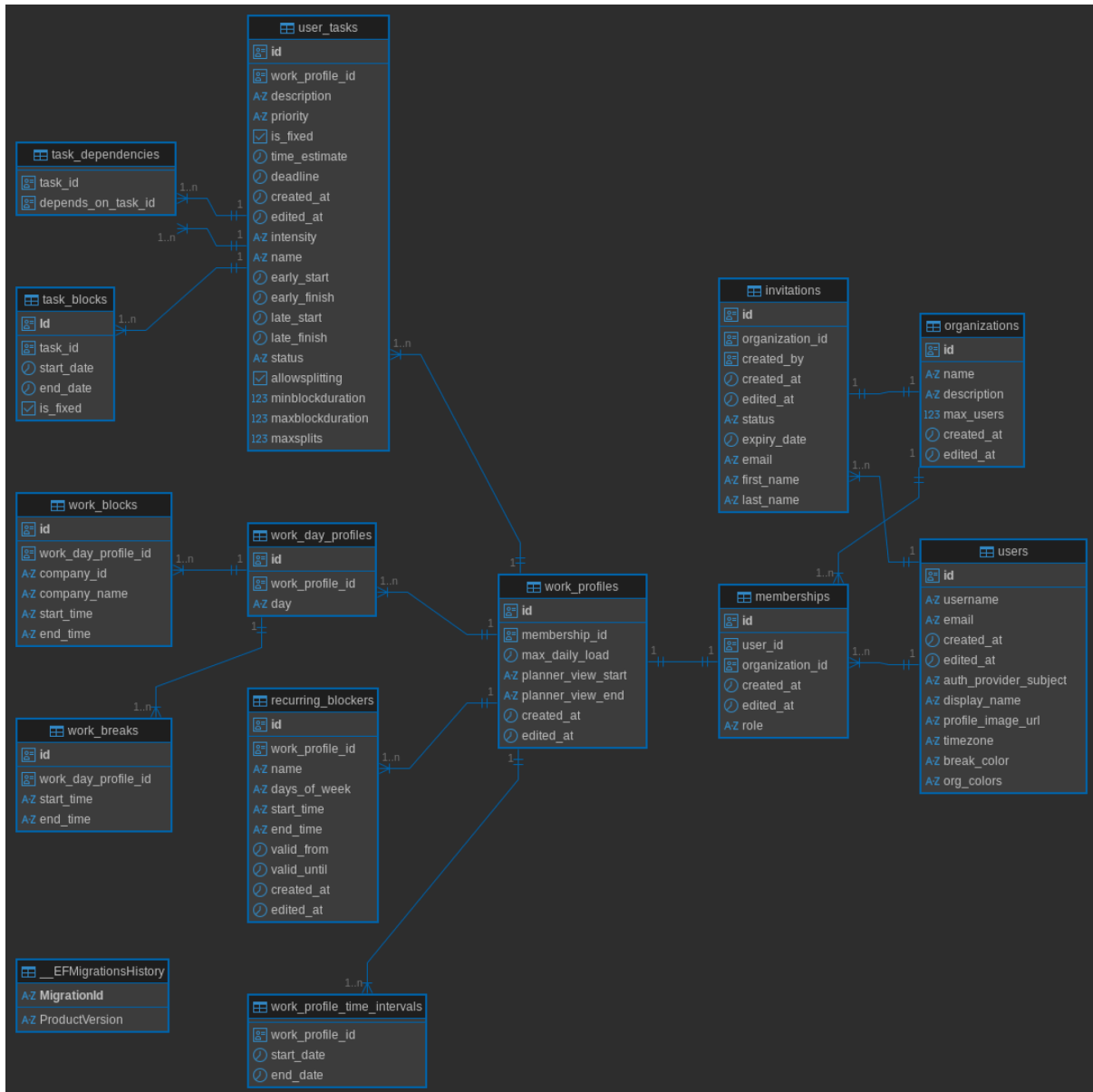
3.2.1 ER-Schema der Datenbank

Die folgende Abbildung zeigt das Entity-Relationship-Schema der zugrunde liegenden Datenbank (lokale Entwicklungsumgebung).

Während das UML-Klassendiagramm die fachlichen Objekte und deren Beziehungen modelliert, stellt das ER-Schema die konkrete relationale Struktur der Datenbank dar. Die Entitäten werden als Tabellen abgebildet, Beziehungen werden über Fremdschlüssel realisiert.

Beispielsweise wird die Beziehung zwischen Benutzer und Organisation über die Tabelle „Memberships“ modelliert.

Das ER-Schema dient als Grundlage für die Persistenz der im Klassendiagramm definierten Objekte und stellt die Konsistenz zwischen fachlichem Modell und technischer Umsetzung sicher.



3.3 Architektur / Services

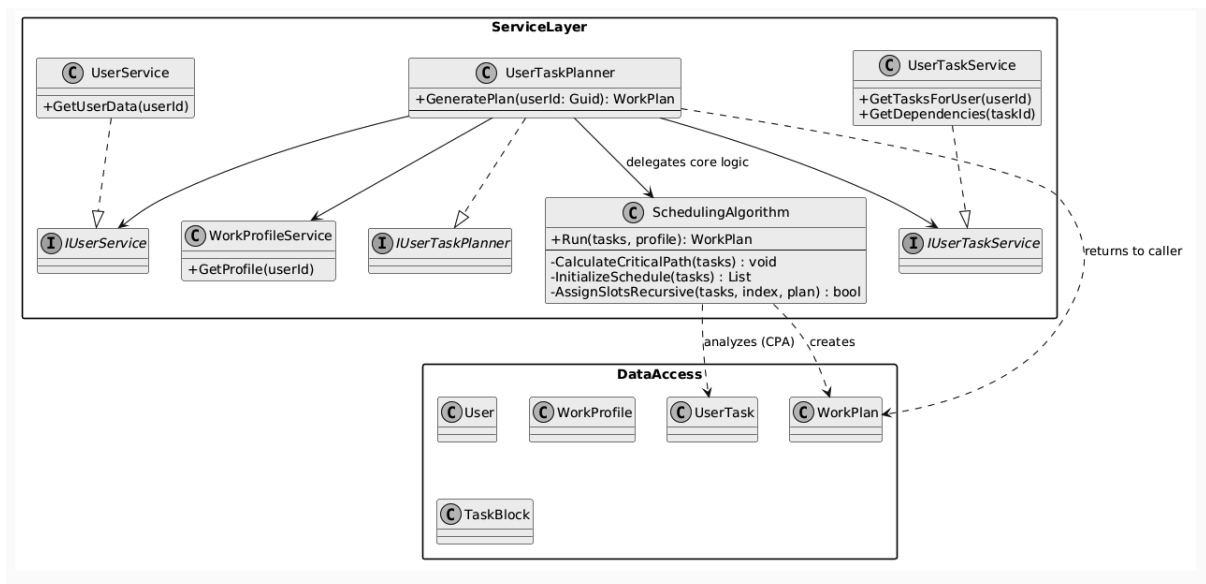
Während das statische Modell die Struktur der Daten definiert, beschreibt der Service Layer die funktionale Logik des Systems. Dieser Aufbau ist notwendig, um die Geschäftslogik strikt von der Datenhaltung zu trennen (Separation of Concerns).

Der UserTaskPlanner fungiert hierbei als zentrale Orchestrierungskomponente: Er ist dafür verantwortlich, die notwendigen Entitäten (Aufgaben, Abhängigkeiten und Nutzerprofile) über die jeweiligen Services zu laden und das Ergebnis der Planung in einem WorkPlan-DTO für das Frontend bereitzustellen.

Die eigentliche Intelligenz ist in den SchedulingAlgorithm ausgelagert. Diese Trennung ist entscheidend, da der komplexe, rekursive Algorithmus so isoliert von der Datenbanklogik

getestet und optimiert werden kann. Die algorithmische Umsetzung erfolgt dabei in drei aufeinanderfolgenden Phasen:

1. **Analyse-Phase (CPA):** Mathematische Identifikation von Prioritäten und Engpässen (Kritischer Pfad).
2. **Initialisierungs-Phase:** Abgleich der Aufgaben mit den realen Zeitfenstern des Arbeitsprofils.
3. **Rekursive Planungs-Phase:** Effiziente Suche nach validen Zeitslots mittels Tiefensuche. Durch das im Code verankerte **implizite Backtracking** (Rekursion) wird sichergestellt, dass das System auch bei hoher Auslastung eine valide Lösung findet oder alternative Pfade prüft, ohne dass eine manuelle Konfliktlösung durch den Nutzer nötig ist.



4. Dynamisches Modell

Das dynamische Modell beschreibt, wie die Klassen zur Laufzeit interagieren. Es wird anhand der wichtigsten Anwendungsfälle des MVP dargestellt.

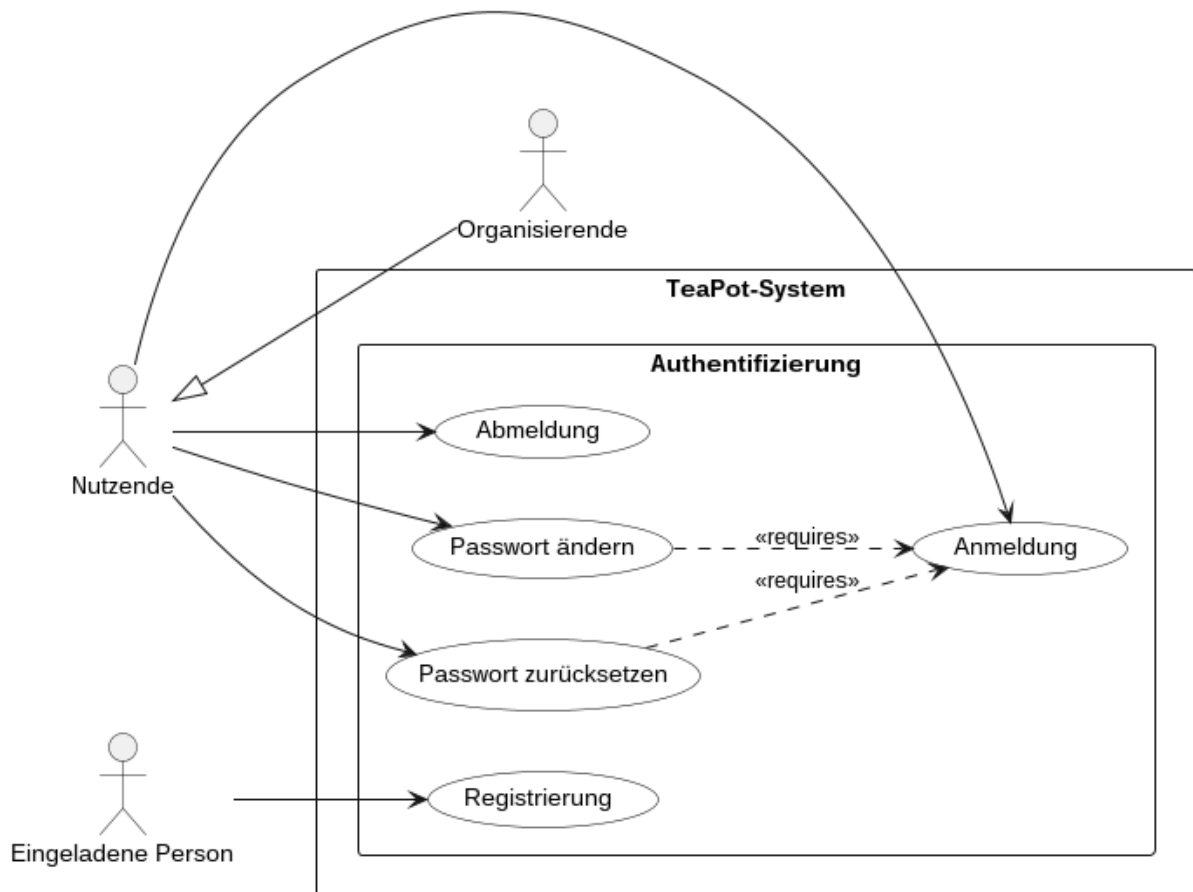
4.1 Use-Case-Diagramme des dynamischen Modells

4.1.1 Authentication Use Cases

Dieses Diagramm zeigt die Anwendungsfälle rund um die Authentifizierung. Nutzende können sich anmelden, abmelden, ihr Passwort ändern oder zurücksetzen. Eine eingeladene Person kann sich registrieren.

Die Beziehungen verdeutlichen außerdem, dass bestimmte Aktionen nur in Verbindung mit einem bestehenden Login-Kontext möglich sind.
Das Diagramm beschreibt damit den Einstieg und die Zugriffssicherung des Systems.

TeaPot - Authentication Use Cases



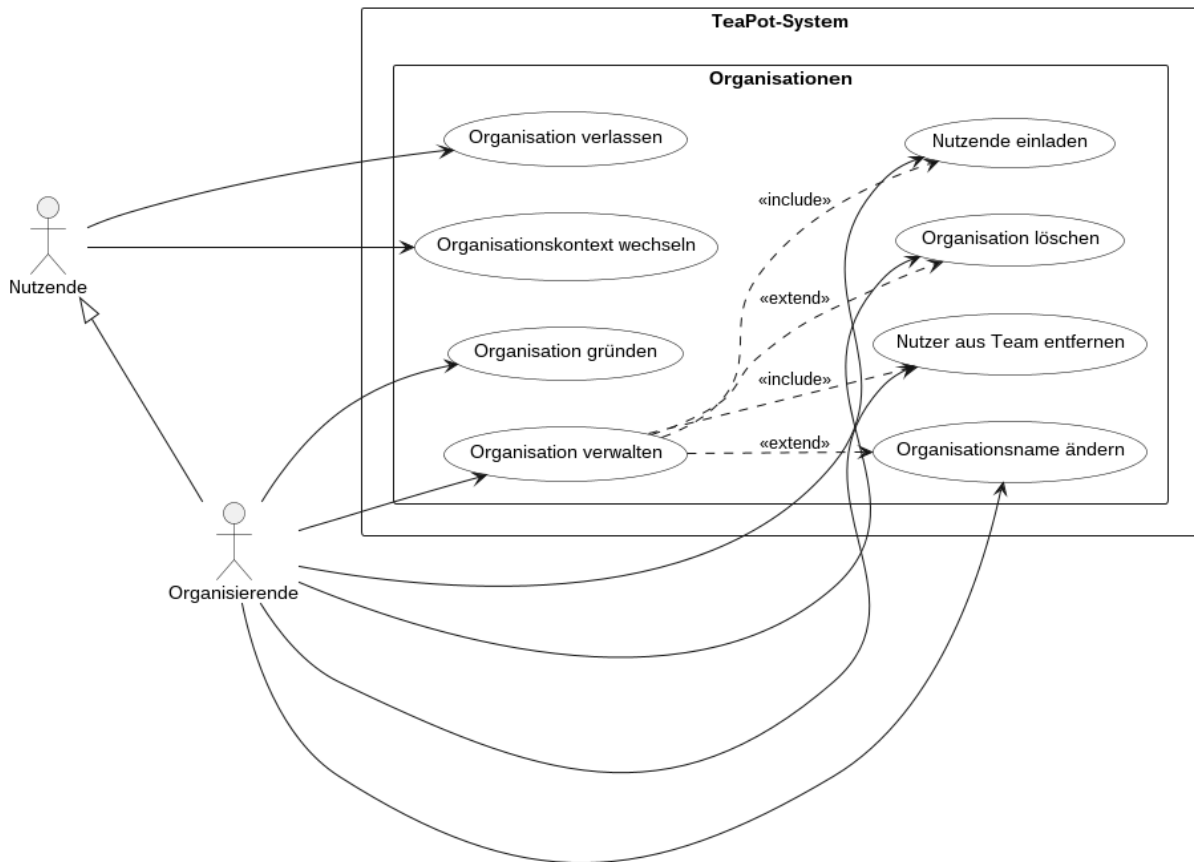
4.1.2 Organization Use Cases

Hier werden die Anwendungsfälle zur Verwaltung von Organisationen dargestellt. Organisierende können Organisationen gründen, verwalten, Mitglieder einladen, entfernen, umbenennen oder löschen.

Normale Nutzende können Organisationen verlassen und den Organisationskontext wechseln.

Die include- und extend-Beziehungen zeigen, dass die Organisationsverwaltung aus mehreren Teilfunktionen besteht.

TeaPot - Organization Use Cases



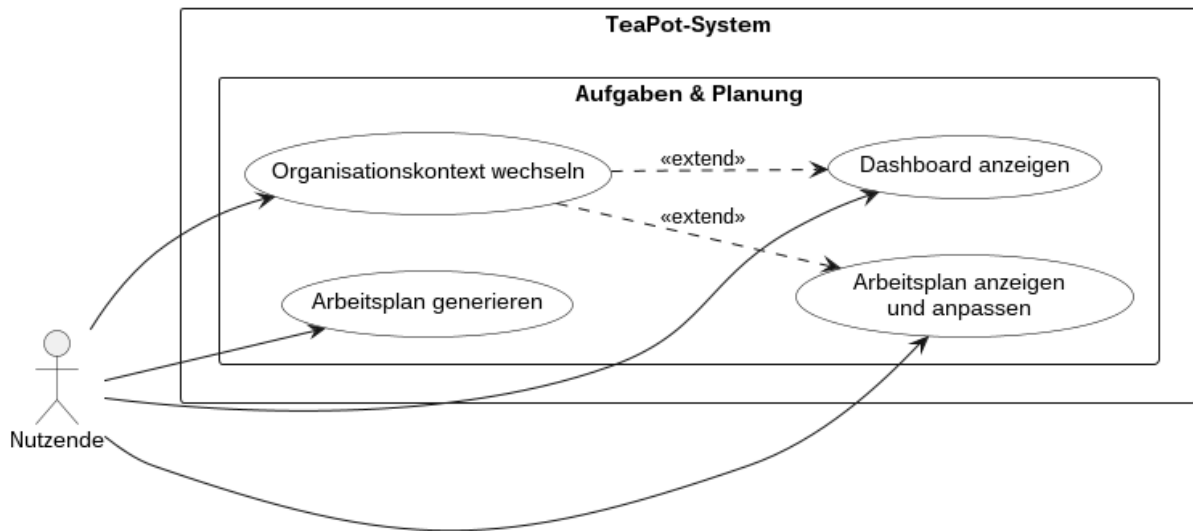
4.1.3 Planning Context Use Cases

Dieses Diagramm beschreibt den Nutzungskontext rund um Dashboard und Planung. Nutzende können das Dashboard aufrufen, den Organisationskontext wechseln, einen Arbeitsplan generieren und diesen anzeigen oder anpassen.

Der Kontextwechsel erweitert bestehende Nutzungssituationen, da sich Inhalte je nach aktiver Organisation ändern können.

Damit wird der Zusammenhang zwischen Navigation, Kontext und Planungsfunktionen deutlich.

TeaPot - Planning Context Use Cases



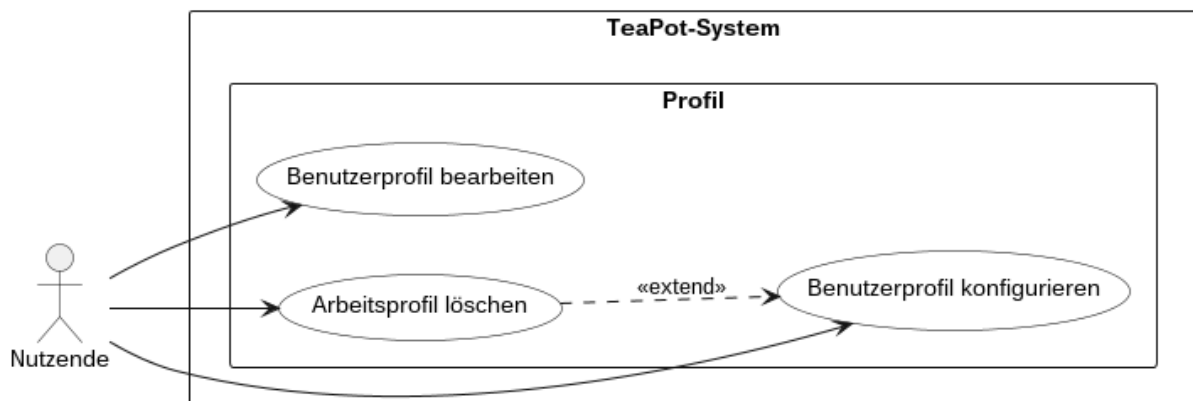
4.1.4 Profile Use Cases

Das Profil-Diagramm zeigt die Funktionen zur Verwaltung persönlicher Einstellungen. Nutzende können ihr Benutzerprofil bearbeiten, ihr Arbeitsprofil konfigurieren und ein Arbeitsprofil löschen.

Die Beziehung zwischen Konfiguration und Löschung macht deutlich, dass beide Funktionen fachlich zusammengehören.

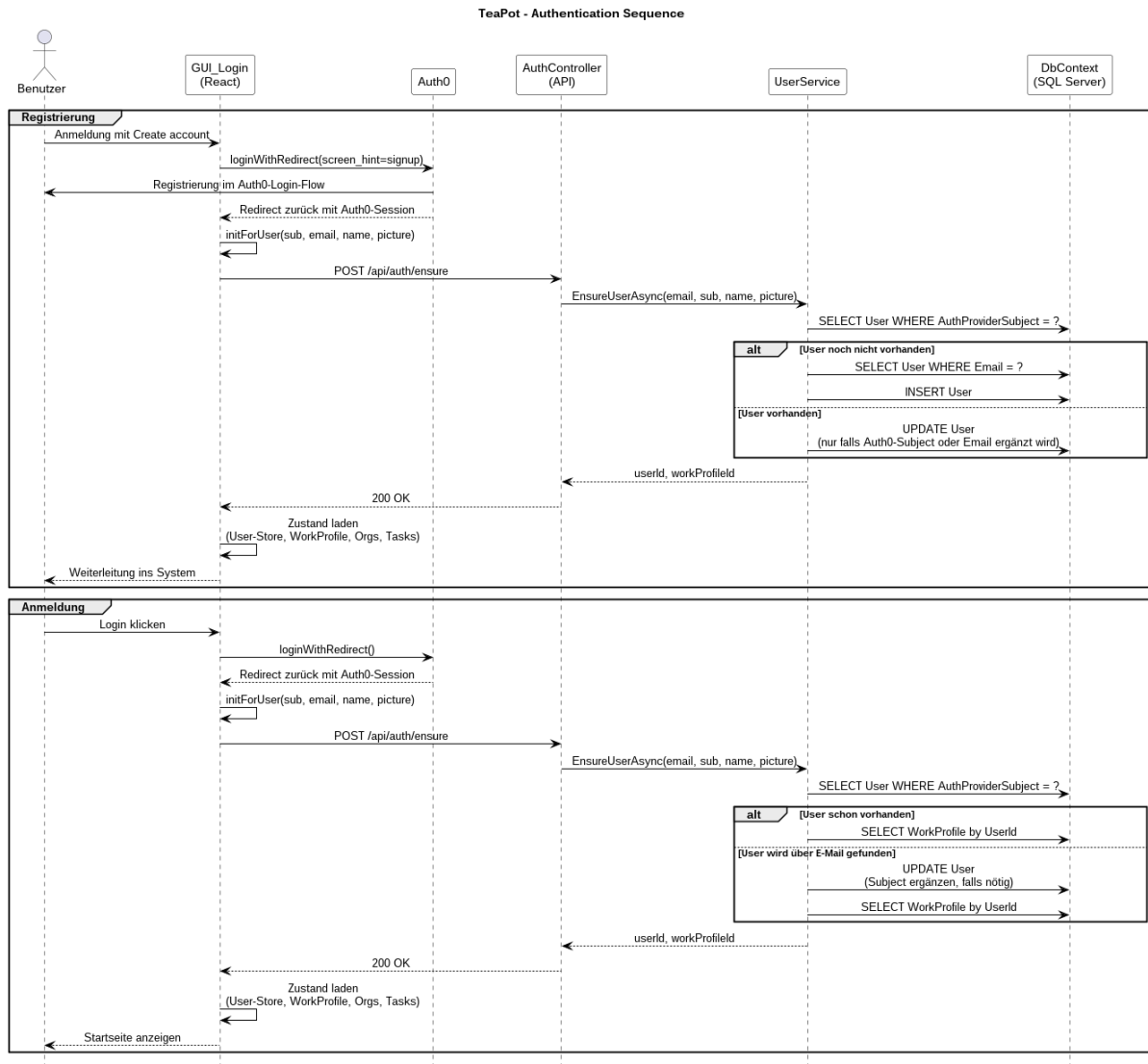
Das Diagramm fokussiert damit die Individualisierung des Systems auf Nutzendenebene.

TeaPot - Profile Use Cases

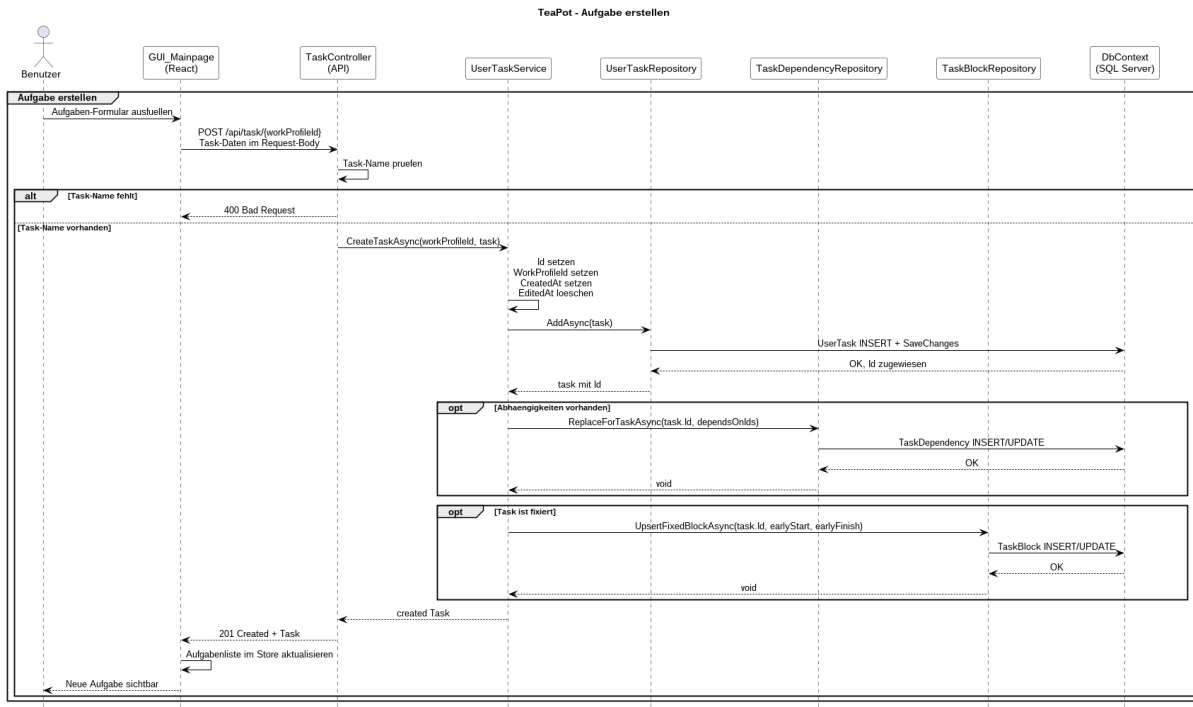


4.2 Sequenzdiagramm: Benutzer registrieren und anmelden

Dieser Ablauf zeigt die Interaktion bei der Erstregistrierung und dem anschließenden Login.

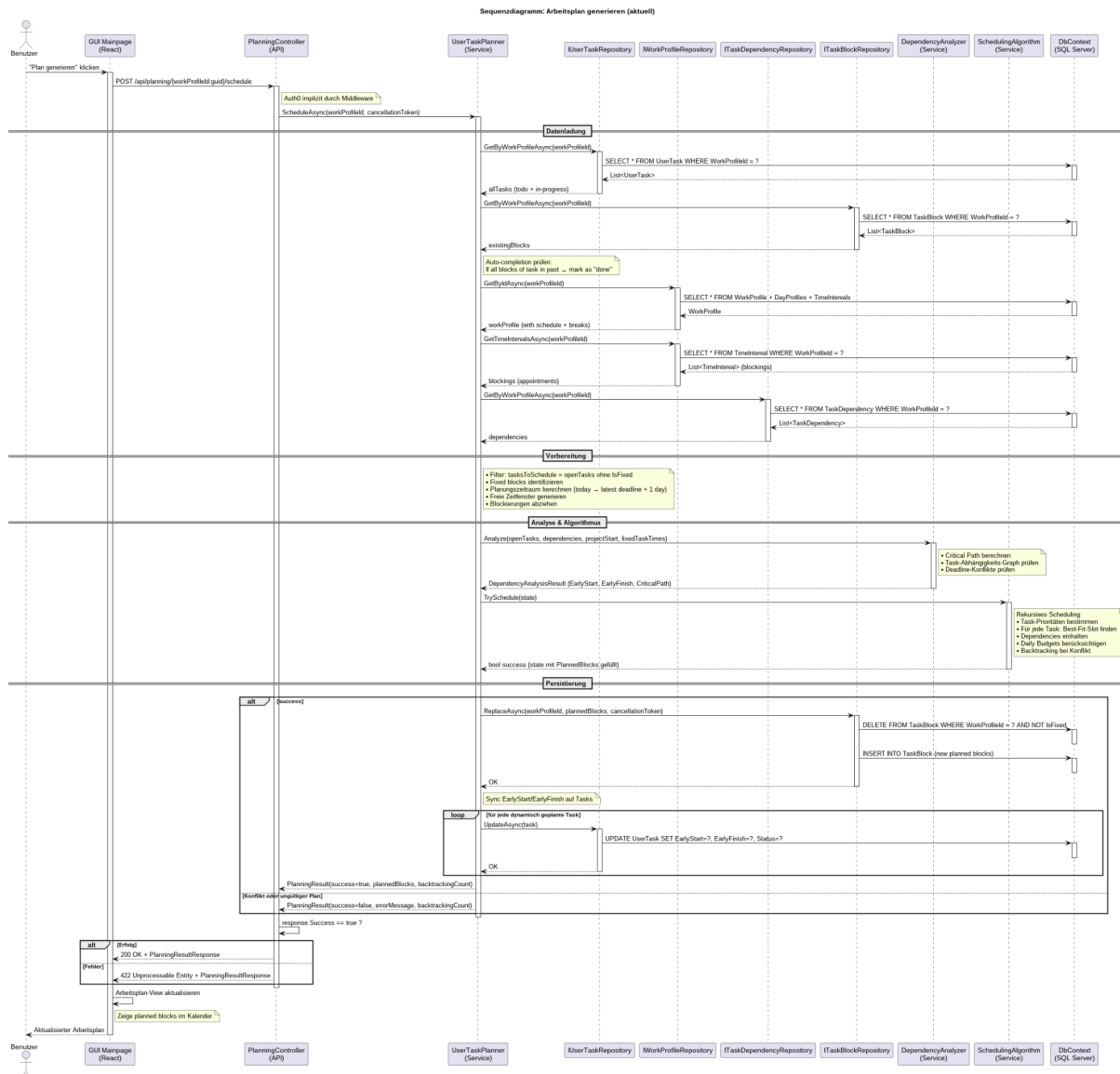


4.3 Sequenzdiagramm: Aufgabe erstellen



4.4 Sequenzdiagramm: Arbeitsplan generieren

Dies ist der komplexeste und für das System charakteristische Ablauf.



Begründung der Methoden:

Aus diesem Szenario ergeben sich folgende Methoden, die den Klassen zugewiesen werden:

- `PlanningController.GeneratePlan()` – delegiert an Service
- `PlanningService (via IUserTaskPlanner) → PlanUserTasks(user: User, forcePlan: bool): List<UserTask>`
- `SchedulingAlgorithm`: weist Aufgaben konkrete `TimeInterval`-Slots zu (Reihenfolge nach Priorität, Deadline, Abhängigkeiten)

4.4.1 Beschreibung des Planungsalgorithmus

Dieser Abschnitt beschreibt den Algorithmus zur automatischen Generierung eines Arbeitsplans (`SchedulingAlgorithm`). Die Darstellung folgt der tatsächlichen internen Logik des Systems und ist in klar getrennte Phasen unterteilt.

Der SchedulingAlgorithm läuft im Backend (.NET 10) im Service-Layer und erhält seine Daten über Repositories bzw. Services. Eine Persistenz der erzeugten Pläne erfolgt über Entity Framework Core in PostgreSQL; für automatisierte Tests wird der InMemory-Provider verwendet.

Ziel des Algorithmus ist es, Aufgaben unter Berücksichtigung von Abhängigkeiten, Prioritäten, Deadlines, Intensität sowie individueller Arbeitsprofile realistisch und effizient in verfügbare Zeitfenster einzuplanen.

1. Analyse der Aufgaben und kritischer Pfad

Zu Beginn werden alle Aufgaben und ihre Abhängigkeiten als gerichteter Graph betrachtet. Dabei wird zunächst geprüft, ob zyklische Abhängigkeiten vorliegen, da in diesem Fall keine gültige Planung möglich ist.

Anschließend erfolgt eine topologische Sortierung der Aufgaben. Auf dieser Grundlage werden frühestmögliche Start- und Endzeitpunkte berechnet. Diese Analyse dient insbesondere dazu, kritische Aufgabenketten zu identifizieren und frühzeitig zu erkennen, ob Deadlines prinzipiell eingehalten werden können.

Falls bereits in dieser Phase festgestellt wird, dass eine konsistente Planung aufgrund von Abhängigkeiten oder Deadlines nicht möglich ist, wird der Planungsprozess abgebrochen und ein entsprechender Hinweis erzeugt.

2. Initialisierung des Planungsraums

Nach der erfolgreichen Analyse werden alle relevanten Daten geladen. Dazu gehören insbesondere die offenen Aufgaben mit ihren Attributen (Restdauer, Priorität, Deadline, Intensität), das Arbeitsprofil der Nutzenden (Arbeitszeiten, maximale tägliche Belastung) sowie vorhandene feste Termine.

Auf Basis dieser Informationen werden Arbeitszeitfenster erzeugt und feste Termine als Blockierungen eingetragen. Dadurch entsteht eine strukturierte Darstellung aller freien Zeitslots, die für die Planung verwendet werden können.

Zusätzlich werden interne Zustände initialisiert, insbesondere die verbleibende Restdauer der Aufgaben sowie Hilfsvariablen zur Steuerung der Planung.

3. Rekursive Planung der Aufgaben

Die eigentliche Einplanung erfolgt rekursiv. In jedem Schritt wird eine geeignete Aufgabe ausgewählt. Dabei werden insbesondere folgende Kriterien berücksichtigt:

- Einhaltung von Abhängigkeiten (eine Aufgabe kann erst geplant werden, wenn alle Vorgänger abgeschlossen sind)
- Deadlines
- Priorität
- Intensität

Für die ausgewählte Aufgabe wird ein geeigneter freier Zeitslot gesucht. Dabei wird geprüft, ob die Aufgabe vollständig oder teilweise in den Slot eingeplant werden kann.

Ist der Slot kleiner als die verbleibende Restdauer der Aufgabe, wird ein Teilblock geplant und die Restdauer entsprechend reduziert. Dieser Mechanismus ermöglicht eine flexible Aufteilung von Aufgaben über mehrere Zeitintervalle hinweg.

Nach erfolgreicher Einplanung wird der Algorithmus mit dem aktualisierten Zustand erneut aufgerufen, bis alle Aufgaben verarbeitet wurden.

4. Behandlung von Konflikten

Falls für eine Aufgabe kein geeigneter Zeitslot gefunden wird, kehrt der Algorithmus in einen vorherigen Zustand zurück und versucht eine alternative Einplanung.

Dieses Verhalten ergibt sich direkt aus der rekursiven Struktur des Algorithmus. Ein separates Backtracking-System oder ein expliziter Stack zur Verwaltung von Zuständen ist nicht erforderlich.

5. Abschluss und Validierung

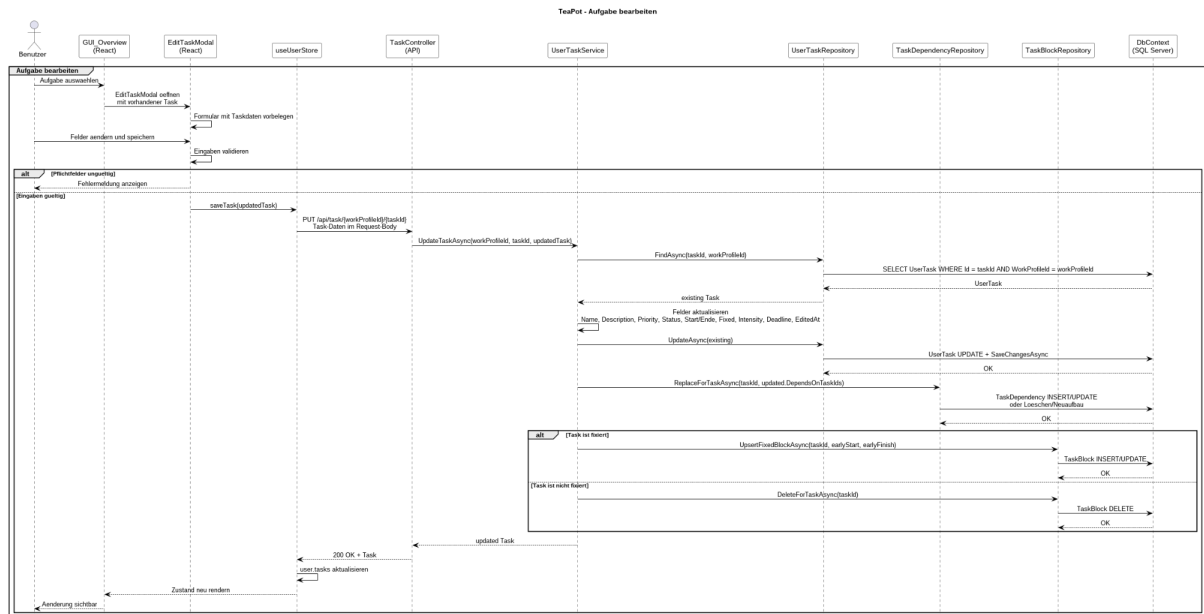
Nach Abschluss der Planung wird der erzeugte Arbeitsplan überprüft. Dabei wird insbesondere sichergestellt, dass:

- keine Zeitintervalle sich überschneiden
- Deadlines eingehalten werden
- definierte Arbeitszeitgrenzen nicht überschritten werden

Ist der Plan gültig, wird er über Entity Framework Core (Produktivbetrieb: PostgreSQL, Testbetrieb: InMemory-Provider) gespeichert und anschließend an das Frontend übergeben. Andernfalls wird ein Konflikt gemeldet und die Planung entsprechend angepasst oder abgebrochen.

Zusammenfassend basiert der Planungsalgorithmus auf einer vorgelagerten Analyse der Aufgabenstruktur, einer klaren Initialisierung des Planungsraums sowie einer rekursiven Einplanung der Aufgaben. Dadurch wird eine flexible und gleichzeitig nachvollziehbare Planung ermöglicht, ohne ein separates Backtracking-Verfahren implementieren zu müssen.

4.5 Sequenzdiagramm: Aufgabe bearbeiten



4.6 Identifizierte Methoden je Klasse

Aus den Sequenzdiagrammen lassen sich folgende Methoden systematisch ableiten:

Klasse	Methode	Auslöser
AuthController	EnsureUser()	Login / Auth0-Flow
AuthController	RegisterAsync()	Registrierung
TaskController	CreateTask()	Aufgabe erstellen
TaskController	GetTask()	Aufgabenübersicht
TaskController	UpdateTask()	Aufgabe bearbeiten
TaskController	DeleteTask()	Aufgabe löschen
PlanningController	Schedule()	Arbeitsplan generieren
UserService	EnsureUserAsync()	Benutzer beim Login sicherstellen / anlegen

UserService	GetProfileAsync()	Profil abrufen
UserService	UpdateProfileAsync()	Profil bearbeiten
UserTaskService	GetUserTasks(userId)	Aufgabenliste, Planung
UserTaskService	AddUserTask(userId, dto)	Aufgabe erstellen
UserTaskService	UpdateUserTask(userId, taskId, dto)	Aufgabe bearbeiten
UserTaskService	DeleteUserTask(userId, taskId)	Aufgabe löschen
UserTaskPlanner	PlanUserTasks(user, forcePlan)	Plan generieren
SchedulingAlgorithm	GenerateSchedule(user)	Aufgaben einplanen
WorkProfileService	UpdateWorkProfile()	Arbeitsprofil konfigurieren
TaskBlockRepository	ReplaceAsync(workProfileId, plannedBlocks)	Generierten Plan speichern
MembershipService	RemoveMembershipAsync()	Nutzer aus Team löschen

5. API-Spezifikation

Methode	Pfad	Auth	Input/Output DTOs	Status-Codes
Get Orgs	/api/organization/by-user-email?email=...	OAuth	OrgIdList, OrgDTOList	200, 400, 404, 500
Join Org	/api/invitation/{invitationId}/accept	OAuth	invitationId	200, 400, 404, 500

Leave Org	/api/membership/leave	OAuth	OrgId	200, 400, 404, 500
CreateTask	/api/task/{workProfileId}	OAuth	TaskDTO	201, 400
GetTasks	/api/task/{workProfileId}	OAuth	TaskList	200, 400, 404, 500
DeleteTask	/api/task/{workProfileId}/{taskId}	OAuth	TaskDTO	204, 404
UpdateTask	/api/task/{workProfileId}/{taskId}	OAuth	UserTask / UserTask	200, 404
ScheduleTasks	/api/planning/{workProfileId}/schedule	OAuth	ScheduleDTO	200, 422

6. Design Patterns

Das System setzt folgende Entwurfsmuster ein:

Pattern	Anwendungsort	Nutzen
Repository Pattern	UserService, UserTaskService, OrganizationService, WorkProfileService greifen über Repositories auf die Daten zu	Kapselt den Datenbankzugriff und entkoppelt Geschäftslogik von der Persistenzschicht
Dependency Injection	Alle Services erhalten ihre Abhängigkeiten per Konstruktor und werden im DI-Container registriert	Erhöht Modularität, Austauschbarkeit und Testbarkeit

Service Layer Pattern	UserService, UserTaskService, OrganizationService, WorkProfileService, UserTaskPlanner	Trennt Geschäftslogik von Controller und Datenschicht
Strategy Pattern	IUserTaskPlanner, UserTaskPlanner, SchedulingAlgorithm	Erlaubt Austausch des Planungsalgorithmus ohne API-Änderung
DTO Pattern	z. B. UserProfileDto, UpdateUserProfileCommand, OrganizationUserDto, PlanningResultResponse, TaskBlockResponse	Trennt API-Datenstrukturen von internen Domänen- bzw. Persistenzmodellen

7. Frontend-Struktur

Das Frontend basiert auf React 19 mit Vite und TypeScript. Der Entwicklungsserver wird über Vite gestartet, Tests laufen mit vitest. Die Authentifizierung im Client ist über Auth0 integriert. Der globale Zustand wird mit einer Kombination aus Zustand und React Context verwaltet. Der Befehl `npm run dev` startet den Entwicklungsmodus, `npm run build` erzeugt die Produktionsartefakte.

Die React-Anwendung gliedert sich in folgende Hauptkomponenten:

Komponente	Zuständigkeit
GUI_Login	Formular für Login und Registrierung
GUI_Mainpage	Navigations-Hub mit Buttons zu allen Unterbereichen
GUI_Overview	Aufgabenübersicht (Deadlines, anstehende Tasks, Wochenmengen)
GUI_ProfileButton	Profilbild, Name, Signed-In-Status
GUI_MyTeams	Liste aller Teams denen man zugehört
GUI_Planner	Kalenderübersicht der Aufgaben und Aufgabenverwaltung

GUI_Tasks	Listenübersicht der Aufgaben und Aufgabenverwaltung
GUI_Settings	Seite der Profileinstellungen

Der API-Zugriff erfolgt über einen zentralen **API-Service** im Frontend, der JWT-Token automatisch in den Authorization-Header einfügt. State wird per **React Context API** oder Redux Toolkit verwaltet (User State, Task State, Plan State).

8. Deployment und Entwicklungsinfrastruktur

Die Bereitstellung erfolgt via Docker-Container auf einem VPS oder Cloud-Anbieter (z. B. Azure oder AWS). Das Docker-Image basiert auf .NET 10 (Production base image) und läuft im Container auf Port 8080.

Die CI/CD-Pipeline (GitHub Actions) automatisiert Build, Test und Deployment:

1. CI/CD: GitHub Actions mit zwei Workflows: Backend-Tests (dotnet test auf .NET 10) und Frontend-Tests (vitest mit Node 22). Die Workflows sind in backend-tests.yml (Backend) und frontend-tests.yml (Frontend) definiert.
2. Tests & Coverage: Backend-Unit-Tests laufen mit dotnet test unter Verwendung von NUnit in den Testprojekten; Coverlet ist als Collector eingebunden. Frontend-Tests werden mit vitest ausgeführt. Das Designmodell kann gewünschte CI-Anforderungen (z. B. Coverage-Schwellen) dokumentieren; die aktuelle Workflow-Konfiguration erzwingt jedoch keine feste Coverage-Grenze.

Versionierung erfolgt nach **Semantic Versioning** (MAJOR.MINOR.PATCH). Bei kritischen Fehlern nach Deployment kann auf das letzte stabile Docker-Image zurückgerollt werden.